QARABAĞ UNİVERSİTETİ

MÜHƏNDİSLİK FAKÜLTƏSİ - KOMPÜTER MÜHƏNDİSLİYİ

LABORATORİYA HESABATI

Xətti Əlaqəli Siyahı (Singly Linked List) və Əsas Əməliyyatlar

Tələbə:	Röyal Yahyayev	**Qrup:**	KOM25
Fənn:	Verilənlər Strukturu	**Dil:**	C#

1. MƏQSƏD VƏ ƏSAS ANLAYIŞLAR

Bu laboratoriya işində məqsəd dinamik yaddaş strukturuna malik **Xətti Əlaqəli Siyahı (Singly Linked List)** üzərində icra olunan əsas alqoritmləri mənimsəməkdir. C# dilində hər bir düyün (Node) məlumat sahəsi və növbəti düyünə göstəricidən (pointer/next) ibarətdir. Kod təkrarının qarşısını almaq üçün hesabatda yalnız əsas məntiq blokları qeyd edilmiş, hər bir tapşırığın nəticəsi vizual olaraq göstərilmişdir.

2. TAPŞIRIQLAR (1 - 3)

Tapşırıq 1: Siyahının Yaradılması

Düyünlər ardıcıl bağlanır və göstərici vasitəsilə sonsuzluğadək (null-a qədər) ekrana çıxarılır.

```
Node head = new Node(10);
head.next = new Node(20);
head.next.next = new Node(30);
```

Tapşırıq 2: Əvvələ Element Əlavə Etmə

Yeni düyünün `next` göstəricisi mövcud `head`-ə bağlanır, sonra baş göstərici yenilənir.

```
Node newNode = new Node(10);
newNode.next = head;
head = newNode;
```

Tapşırıq 3: Sona Element Əlavə Etmə

Siyahının sonuncu düyününə (next == null olan yerə) qədər gedilir və yeni düyün ora bağlanır.

```
Node current = head;
while (current.next != null) { current = current.next; }
current.next = new Node(30)
```

3. TAPŞIRIQLAR (4 - 6)

Tapşırıq 4: Ortaya Element Əlavə Etmə

Müəyyən olunmuş düyündən sonra yeni element daxil edilir. Əvvəlcə yeni düyünün `next` göstəricisi sonrakı düyünə mənimsədilir, daha sonra əvvəlki düyün yenisinə bağlanır.

```
Node newNode = new Node(30);
newNode.next = head.next.next;
head.next.next = newNode;
```

Tapşırıq 5: Elementin Silinməsi

Silinmək istənən düyündən əvvəlki düyünün `next` göstəricisi birbaşa silinəcək düyündən sonrakı elementi göstərir. Beləliklə, aradakı düyün yaddaş zəncirindən çıxarılır.

```
// 20-den sonra gelen 30 elementi zencirden cixarilir
head.next.next = head.next.next.next;
```

Tapşırıq 6: Axtarış Əməliyyatı (Search)

Siyahının başlanğıcından sonuna qədər hər bir düyünün dəyəri (data) verilmiş hədəf dəyərlə müqayisə edilir. Tapıldıqda dövr dayandırılır (O(N) zaman mürəkkəbliyi).

```
int target = 30;
while (current != null) {
    if (current.data == target) { found = true; break; }
    current = current.next;
}
```

4. TAPŞIRIQLAR (7 - 9) VƏ YEKUN

Tapşırıq 7: Uzunluğun Hesablanması

Siyahı üzrə irəliləyərək hər bir adımlamada sayğac (count) dəyişəni 1 vahid artırılır və ümumi düyünlərin sayı müəyyən edilir.

```
int count = 0;
while (current != null) {
    count++;
    current = current.next;
}
```

Tapşırıq 8: Tərsinə Çevrilmə (In-Place Reverse)

Yenidən yaddaş ayırmadan (təzə siyahı yaratmadan), mövcud düyünlərin `next` göstəriciləri əks istiqamətə yönləndirilir.

```
Node prev = null, current = head, nextNode = null;
while (current != null) {
    nextNode = current.next;
    current.next = prev;
    prev = current;
    current = nextNode;
}
head = prev;
```

Tapşırıq 9: Siyahının Sıfırlanması (Clear List)

Başlanğıc göstəricisi (head) `null` olaraq mənimsədilir. C# Garbage Collector avtomatik olaraq digər qovşaqları yaddaşdan silir.

```
head = null;
Console.WriteLine("Siyahi temizlendi. Netice: None");
```

Yekun Nəticə: Tapşırıqlar 1-dən 9-a qədər uğurla tətbiq edildi. Əlaqəli siyahının strukturu, yaddaş idarəetməsi, göstəricilərin yenilənməsi və in-place alqoritmləri praktiki olaraq mənimsənildi.